AccessWindow / netproxy — Study Guide

> Open `packet_flow.html` in a browser alongside this guide. It animates the exact journey described in §5 (allowed site, blocked site, temporary unblock, content filter). Run the project first (see `README_ACCESSWINDOW.md`) so you can demo every claim below.

---

1. The 30-second explanation

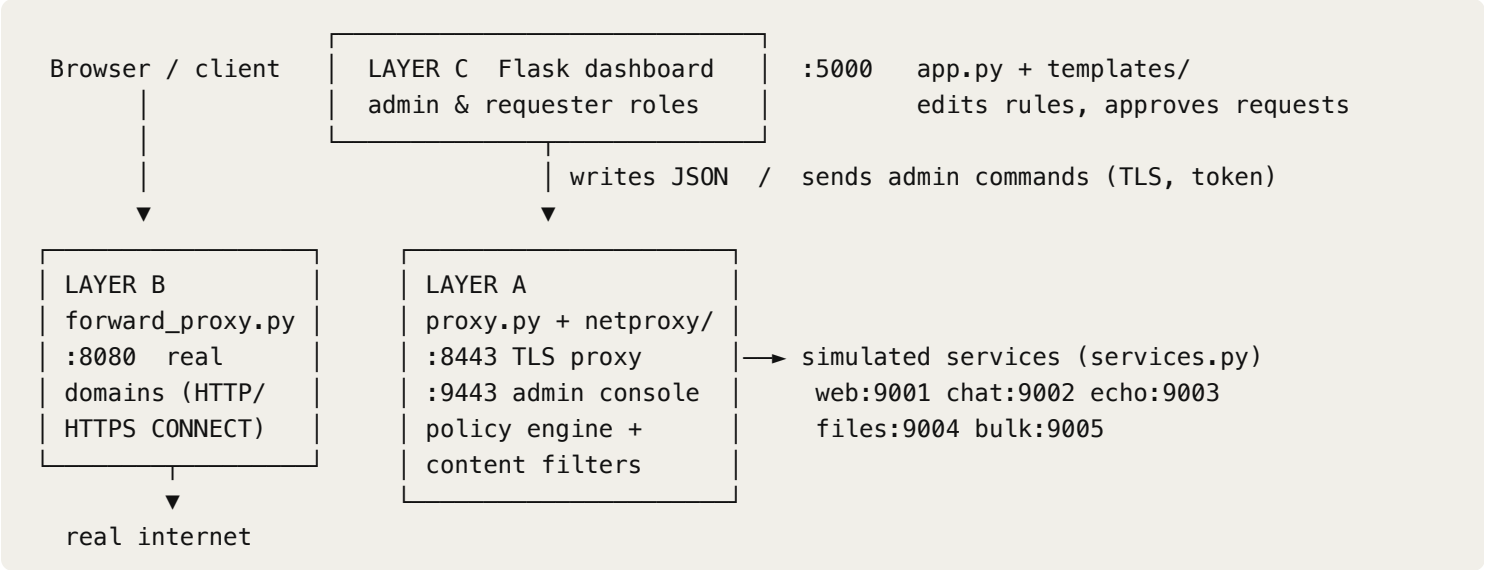
This project is a **content-filtering proxy with an approval workflow**.

- A **proxy** is a middleman. Instead of the browser talking straight to a website, it talks to the proxy, and the proxy decides whether to pass the traffic on.
- The proxy decides using an ordered list of **rules** ("deny everyone → youtube.com"). The **first rule that matches wins**; if none match, a **default action** applies.
- A **web dashboard (Flask)** lets an **administrator** manage the rules and lets a **requester** ask for temporary access. The admin approves, a time-limited *allow* rule is inserted at the top of the list, and it **expires by itself**.

If you remember one sentence: *"An ordered, first-match-wins rule list is evaluated for every connection; the dashboard edits that list, and temporary access is just a rule with an expiry time."*

---

2. Architecture — three layers



Layer	What it filters	How	Policy file
A — `netproxy` (the original academic proxy)	Connections to *simulated* services; then **the content** of the stream	TLS, identity from client certificate, rule engine, keyword/regex/MIME filters, rate limits	`config/policy.json`
B — `forward_proxy.py`	**Real public domains** such as `youtube.com`	Standard HTTP proxy; HTTPS handled via `CONNECT`, filtered by **hostname only**	`config/real_website_policy.json`
C — Flask dashboard	Nothing directly — it is the *control plane*	Reads/writes policy, approves requests, shows logs	both of the above

Why two proxies? Layer A is a full TLS proxy for a closed lab (only the services registered in the policy can be reached). Layer B is a lightweight proxy so a normal browser can be filtered on real websites.

---

3. File map — which file does what

Entry points (things you run)

File	Purpose
`services.py`	Starts the five **simulated back-end servers** with the raw `socket` API (web, chat, echo, files, bulk). They trust everyone — access control is the proxy's job.
`proxy.py`	Starts **Layer A**: loads `config/proxy.json` + `config/policy.json`, starts `ProxyServer`, the admin console, and the grant-expiry worker.
`forward_proxy.py`	Starts **Layer B** (real-domain proxy on `127.0.0.1:8080`). With `--block-page` it also shows a real block page for blocked HTTPS sites.
`blockpage.py`	The block page: a dedicated local CA, per-host certificates made with `openssl`, and the "why / by whom" page served over TLS. Only blocked hosts are intercepted.
`app.py`	**Layer C** — the Flask dashboard (`127.0.0.1:5000`).
`client.py`, `adminctl.py`	Command-line client (reach services through the proxy) and admin shell (change policy live).
`gen_certs.py`	Generates the CA, the proxy certificate and client certificates (for TLS / mutual TLS).
`demo.py`, `benchmark.py`	Scripted demo and performance measurements.

The `netproxy/` package (Layer A internals)

Module	Responsibility
`policy.py`	**The policy engine.** Parses/validates the JSON policy into immutable objects (`Rule`, `Schedule`, `UserMatcher`, `ContentFilter`, `Limits`, `Policy`); `Policy.evaluate()` does first-match; `PolicyStore` swaps in new versions atomically, watches the file, and notifies listeners.
`server.py`	**The proxy itself.** `ProxyServer` accepts connections; one `ClientSession` thread per client runs: TLS handshake → admit → read request → policy check → connect to back end → relay/inspect.
`filters.py`	**Content inspection.** `inspect_bytes()` (keyword/regex), `check_mime()`, and `StreamInspector` (re-assembles a TCP byte stream into lines before scanning).
`httpmsg.py`	Strict, minimal HTTP/1.1 parser (request/response heads, bodies, block page). Rejects ambiguous framing to prevent request smuggling.
`registry.py`	Live connection table, per-user connection **limits**, counters, and the access log. `revalidate()` re-checks open connections after a policy change.
`control.py`	The **admin console** (a separate TLS listener on `127.0.0.1:9443`, token-authenticated): `BLOCK`, `RULE`, `FILTER`, `KILL`, `STATUS`, `LOG`…
`tlsutil.py`	Builds TLS contexts; extracts the client certificate's Common Name = the user's **identity**.
`clientlib.py`	Client-side library used by `client.py`, `adminctl.py`, the dashboard (`AdminClient`) and the tests.
`accesswindow.py`	Background thread that **expires temporary grants** for the simulated services.
`config.py`, `testbed.py`	Loads `proxy.json`; starts the whole stack in-process for tests.

The dashboard (Layer C)

File	Purpose
`app.py`	Flask routes, login, roles, CSRF, request/approve/revoke logic, website-rule management.
`templates/base.html`	Shared layout: sidebar, account menu, proxy-status pill + glowing dot.
`templates/overview.html`, `requests.html`, `grants.html`, `activity.html`, `browse.html`, `website_access.html`, `login.html`	One page per menu item. `_icons.html` holds the SVG icon macro.
`static/style.css`, `theme.css`, `app.js`	Original styles; the new palette/typography layer; countdown timers + account-menu behaviour.

Tests (`tests/`)

`test_policy.py` (rule engine), `test_filters.py` (content filtering), `test_integration.py` (whole stack over real sockets), `test_forward_proxy_policy.py` (real-domain matching), `test_accesswindow_roles.py` (role separation), `test_browse.py` (Browse page and its safeguards), `test_blockpage.py` (block page over real TLS), `test_block_link.py` (request link and login redirect).

Run them with the project's virtual environment (system Python has no Flask): `venv/bin/python -m unittest discover -s tests` → **86 tests, 85 pass, 1 known environment failure** (see Q42). The temporary-unblock feature is exercised through `test_accesswindow_roles.py`, `test_block_link.py` and `test_browse.py`.

---

4. Core concepts you must be able to explain

4.1 Proxy, forward proxy, tunnel

- **Forward proxy**: the *client* is configured to use it. It receives requests meant for other servers.
- For plain HTTP the client sends `GET http://site/path HTTP/1.1` to the proxy (absolute URL).
- For HTTPS the client sends `CONNECT host:443`. If allowed, the proxy opens a TCP connection to the host, replies `200 Connection Established`, and then simply **copies bytes** both ways. The bytes are TLS-encrypted end-to-end, so the proxy **cannot read them** — it only knows the **hostname** from the `CONNECT` line.

4.2 Access Control List (ACL) — first match wins, default action

```
rule 1: allow  bob  → youtube.com      ← temporary, inserted at the top
rule 2: deny   *    → youtube.com
default: allow
```

Rules are checked **top to bottom**; the first one whose conditions all match decides. Order is therefore the whole logic: that's why a temporary *allow* is **inserted at position 0** — it beats the older *deny* beneath it. This is exactly how firewall ACLs behave.

4.3 Default deny vs default allow

- Layer A (`policy.json`): `"default_action": "deny"` — anything not explicitly allowed is refused (safer; "whitelist" model).
- Layer B (`real_website_policy.json`): `"default_action": "allow"` — everything works unless explicitly blocked (a "blacklist" model, practical for a normal browser).

4.4 Identity — who is the user?

- **Layer A:** by **mutual TLS**. The client presents a certificate signed by the project CA; the proxy reads its **Common Name** (`bob`). No certificate → `anonymous`. Rules match on that identity, *not on anything the client merely claims*.
- **Layer B:** HTTP **Basic proxy authentication** (`Proxy-Authorization` header, base64 of `user:password`). If missing/wrong → `407 Proxy Authentication Required`.
- **Dashboard:** username/password form → server-side session cookie.

4.5 Two-stage evaluation (Layer A)

- **Connect stage** — when a client asks to `CONNECT` to a service. Known: user, source IP, service, time.
- **Request stage** — for every HTTP request *inside* an HTTP-inspected tunnel. Now the **method and path** are known too, so rules like `deny /admin/*` can apply.
- A rule that needs a path can't be decided at connect time: a *deny* is skipped (enforced later per request) and an *allow* admits the connection **provisionally**.

4.6 Path normalisation (security!)

`normalize_path()` canonicalises the URL **before** matching. Without it, `/x/../admin`, `//admin` or `/%61dmin` would slip past a rule for `/admin/*` while the back end still serves the admin page. The proxy forwards the **normalised** path, so *what was checked is exactly what is served*.

4.7 Content filtering

Three filter types, three actions:

Type	Matches	Example
`keyword`	words (optionally whole-word, case-insensitive)	F01 `casino`, `gambling`, `betting`
`regex`	patterns	F02 payment-card numbers → `[CARD REDACTED]`
`mime`	`Content-Type` of responses	F04 blocks `application/x-msdownload` (executables)

Action	Effect
`block`	Request refused (`403`) or, on a stream, the **session is terminated**
`redact`	Matched text replaced (e.g. `***`), traffic continues
`log`	Only recorded (audit)

Order inside `inspect_bytes()`: **any block wins outright → then redactions → log filters only record.**

Why `StreamInspector` exists: TCP is a *byte stream* with no message boundaries — one `recv()` may return half a line or ten lines. A keyword can be **split across two packets** (`"confi"` + `"dential\n"`). The inspector buffers bytes and scans only **complete lines**; at the end of a partial line it holds back any tail that could be the start of a blocked keyword. Result: filtering outcome doesn't depend on how TCP happened to segment the data (there is a test, `test_every_segmentation_gives_same_result`).

4.8 Dynamic policy — change rules without restarting

- `Policy` objects are **immutable**. Any change builds a **new** `Policy` (version + 1) and `PolicyStore._swap()` replaces the old one under a lock → a connection thread never sees a half-applied policy.
- A **watcher thread** polls the file's modification time each second and reloads on change. An invalid file is **rejected** and the old policy stays active.
- After every swap, listeners are called — `ProxyServer._on_policy_change` calls `Registry.revalidate()`, which re-evaluates **every open connection** and **kills** the ones now denied. That is what makes the filtering *dynamic*: blocking a user cuts their live session.

4.9 Rate / connection limits

`Registry.admit()` enforces, per identity: a **maximum concurrent connections** count and a **sliding-window** limit on new connections (`deque` of timestamps; old ones are dropped from the left). Excess → `429 Too Many Requests`.

4.10 Concurrency model

- One **thread per client connection** (`ClientSession.run`), started from the accept loop.
- A `BoundedSemaphore(max_connections)` caps the total; over capacity the socket is closed immediately.

- Sockets are non-blocking + `select()`. `Channel.try_recv()` asks the socket first because an `SSLSocket` can hold already-decrypted bytes that `select()` can't see.
- Each session has a `cancel` `threading.Event`; `kill()` sets it and the loop raises `Terminated` within ~0.1 s (`Channel.TICK`).

4.11 Why a browser can't show a proxy's "blocked" message — and TLS interception

For an HTTPS site the browser sends `CONNECT host:443` and only afterwards starts TLS with the site. If the proxy refuses the `CONNECT`, the browser **ignores the refusal's body** and shows its own "site can't be reached" page. The only way to show a real page is to answer the `CONNECT` with `200` and complete the TLS handshake **ourselves**.

That is **TLS interception**: the proxy presents a certificate for the requested host that it signed with its own CA. The browser accepts it only if it **trusts that CA**, so the CA is installed once. This project intercepts **only blocked hosts**; allowed traffic is still an end-to-end encrypted tunnel. `blockpage.py` creates a dedicated CA (`certs/blockpage/ca.crt`), makes a short-lived (30-day) certificate per blocked host with the `openssl` tool (the host name goes in the Subject Alternative Name — modern browsers ignore the common name), and serves the page over TLS 1.2+.

Risk to be able to state: while the CA is trusted, whoever holds `ca.key` can create certificates your machine accepts for *any* site. So the key is `chmod 600` and git-ignored, the CA is dedicated to this demo, and the trust is removed afterwards.

4.12 Service workers, offline pages and `Clear-Site-Data`

YouTube registers a **service worker** — a script in the browser that sits between the page and the network. When the network request fails it serves a cached **offline page** ("Connect to the Internet", with a Downloads link to offline videos). So a blocked YouTube looked "offline" instead of blocked, and its Downloads page still opened.

The block page is a *successful* response from the site's own origin, so it can remove that state: it sends the `Clear-Site-Data: "cache", "storage"` header and runs a small script that unregisters service workers and deletes caches and IndexedDB for the origin. Login cookies are not cleared. `--keep-site-data` turns this off.

4.13 Related domains

A website is often several domains. YouTube **video** comes from `googlevideo.com`, thumbnails from `ytimg.com`. If only `youtube.com` were blocked, the page might fail but an already-playing video would continue. `DOMAIN_FAMILIES` makes a rule for `youtube.com` also cover `googlevideo.com`, `ytimg.com`, `youtu.be` and `youtube-nocookie.com` — for blocks and for temporary grants alike.

---

5. Life of a packet — step by step (matches `packet_flow.html`)

5.1 Layer B: a browser visits `youtube.com` (blocked)

1. Browser (proxy = `127.0.0.1:8080`) sends `CONNECT youtube.com:443`. → `Handler.handle()` reads the request line and headers.
2. **Authenticate** → `Handler.authenticate()` decodes `Proxy-Authorization`; wrong/missing → `407`.

3. **Parse target** → host `youtube.com`, port `443` (for plain HTTP the target is parsed from the absolute URL with `urllib.parse.urlsplit`).
4. **Match rules** → `match_rule(host, user)`: `python for rule in policy["rules"]: if not rule.get("enabled", True) or rule_expired(rule): continue # skip disabled / expired users = rule.get("users", ["*"]) if users and "*" not in users and user not in users: continue # rule is for someone else if domain_hit(host, rule): # domain, subdomain or related domain return rule["action"], rule # FIRST MATCH WINS return policy["default_action"], None` `host.endswith("." + domain)` makes `www.youtube.com` match but **not** `notyoutube.com` (there's a test for this suffix attack). A small `DOMAIN_FAMILIES` table also makes `youtube.com` cover `googlevideo.com`, `ytimg.com`, `youtu.be` and `youtube-nocookie.com` — YouTube *video* is served from `googlevideo.com`, so without this a blocked YouTube would keep playing.
5. **Verdict** → `deny`: `log_event(...)` appends to `logs/real_proxy_access.jsonl`, and the proxy **never opens a socket to YouTube**. Without `--block-page` it sends a plain `403` (the browser then shows its own error). With `--block-page` it answers `200`, does the TLS handshake with a certificate for that host signed by the block-page CA, and serves the "why / by whom" page with a button to the dashboard's request form (§4.11). `allow`: opens `socket.create_connection((host, port))`, replies `200 Connection Established`, and `tunnel()` copies bytes both ways using `select()`.

5.2 Layer A: a student uses the chat service

1. `ProxyServer._accept_loop` accepts the TCP connection → new thread → `ClientSession.run()`.
2. `_handshake()` → TLS handshake; the client certificate's CN becomes `c.user` (`bob`).
3. `registry.admit()` → per-user connection cap and rate limit.
4. `_serve()` → reads the request head (`CONNECT chat HTTP/1.1`); `httpmsg.parse_request_head` validates it.
5. `_connect()` → `policy.resolve_service("chat")`, then `policy.evaluate(Context(user, src, service), stage="connect")`. Denied → `403` with the **rule id and reason** in `X-Proxy-Rule` / `X-Proxy-Reason`.
6. `_open_backend()` → connect to `127.0.0.1:9002`; reply `200 Connection Established`.
7. `_relay()` (inspect mode `line`) → every chunk from the client goes through `StreamInspector.feed()` → `inspect_bytes()` → `_deliver()`:
   - clean → forwarded;
   - `redact` hit → replaced text forwarded;
   - `block` hit → `info.kill()`, notice sent to the client, `Terminated` raised, session ends.

5.3 Layer A with HTTP inspection (`web` service)

`_http_session` → `_http_exchange` parses **every** request: read body → resolve service → `normalize_path` → `policy.evaluate(..., stage="request")` (path/method rules like R04 `/admin/*`, R05 `/social/*` Mon-Fri 09-17) → request filters on URL and body → forward (adding `X-Forwarded-For`, `X-Authenticated-User`, `Via`; dropping `Accept-Encoding` so bodies can be inspected) → response MIME check and body filters → send. Blocks produce an HTML block page via `httpmsg.block_page`.

---

6. The temporary-unblock feature (what *you* built)

Goal: a user asks for a blocked site for a few minutes; it opens, then closes by itself.

```
Bob ── POST /site-request ──► pending request   (nothing changes yet)
Admin ─ POST /request/<id>/approve ──► rule inserted at index 0:
   {"id":"REAL-9F2A…","action":"allow","users":["bob"],"domains":["youtube.com"],
    "temporary":true,"expires_at":<now + minutes*60>,"grant_reason":"…"}
forward_proxy.match_rule() → first match = this allow → traffic flows
time passes → rule_expired() returns True → rule skipped → deny rule matches again
```

Piece	Where	Logic
Request	`app.py · create_site_request()`	Requester only. Domain must be in `blocked_sites_for(user)` — i.e. the **same matcher the proxy uses** (`forward_proxy.match_rule`) says it is *not* currently allowed. Prevents duplicates and requests for unblocked sites.
Approval	`app.py · decide_request()` (`kind == "site"` branch)	Admin only. Inserts the temporary allow rule at the **top**; records history.
Enforcement	`forward_proxy.py · rule_expired()` + `match_rule()`	Compares `expires_at` with `time.time()` **on every request**. Expiry is therefore enforced by the proxy itself — no background job required for the block to return.
Open tunnels	`forward_proxy.py · tunnel(upstream, deadline, host, user)`	The loop wakes every second: it stops at the `deadline`, and it **re-runs `match_rule` for that host and user**, so revoking a grant (or adding a deny) also closes tunnels that are already open — within about a second.
Clean-up	`app.py · purge_expired_real_rules()`	Removes dead rules from the JSON file (cosmetic; correctness doesn't depend on it).
Early end	`app.py · revoke_site_grant()`	Admin deletes the rule immediately.

Design decision to defend: *where is expiry enforced?* — in the data path (the proxy), not only in a cleanup job. If the dashboard crashed, the block would still return on time.

Parallel mechanism (service grants): for the simulated services, approved grants are stored in `config/accesswindow_state.json` and an expiry **worker thread inside `proxy.py`** (`netproxy/accesswindow.py`, 1 s interval) removes the rule via `store.mutate()`. Because that triggers the policy-change listener, `revalidate()` also **kills live connections** that depended on the expired rule.

---

7. The dashboard code (Flask) — what to know

- **Roles:** `ROLES = {"admin": "admin", "bob": "requester", "carol": "requester"}`. `@login_required` checks the session; `@role_required("admin")` returns **403** otherwise — enforced **server-side**, not just by hiding buttons.
- **Passwords:** stored only as hashes (`generate_password_hash(..., method="pbkdf2:sha256")`); verified with `check_password_hash` (constant-time). Demo defaults can be overridden with `ACCESSWINDOW_*_PASSWORD` environment variables.
- **CSRF protection:** a `before_request` hook rejects every `POST` whose form `csrf_token` doesn't match the one stored in the session (`hmac.compare_digest`). Every form includes `{{ csrf_token() }}`.
- **Session cookie:** `HttpOnly` (JS can't read it) and `SameSite=Lax`.
- **Loopback only:** binds to `127.0.0.1` by default so it isn't exposed to the network.

- **Talking to Layer A:** `admin_command()` opens a TLS connection to the admin console (`127.0.0.1:9443`), authenticates with the token and sends commands like `RULE INSERT 1 {json}` / `RULE DEL <id>` / `STATUS`.
- **Talking to Layer B:** just reads/writes `config/real_website_policy.json`; the forward proxy re-reads it on every request, so changes are live. `write_json()` writes to a temp file then `replace()`s it — an **atomic** update so the proxy never reads a half-written file.
- **Browse (`/browse`):** the address is checked with the **same matcher as the proxy** (`forward_proxy.match_rule`) for the signed-in user. Blocked → a block page with the rule, reason and a request form; allowed → the page fetched server-side and shown in a sandboxed, scripts-off frame. It is a server fetching a user-supplied URL, so it is guarded against SSRF (§8). Every attempt is written to the proxy's access log, so it shows in the blocked-site alerts.
- **Request deep link:** the block page's button opens `/requests?domain=<site>` with the site pre-selected. Signed-out visitors are redirected to `/login?next=…` and returned there after login; only relative `next` paths are honoured (no open redirect).
- **Page structure:** `render_page()` merges `dashboard_data()` (role-filtered requests, grants, history) into every template, so the sidebar badge always matches. `base.html` is the shared layout; each page `extends` it.
- **Routes:** `/` overview · `/requests` · `/grants` · `/activity` · `/browse` · `/websites` · `/login` · `/logout` and the POST actions (`/request`, `/site-request`, `/request/<id>/<approve|reject>`, `/grant/<id>/revoke`, `/site-grant/<id>/revoke`, `/websites/add`, `/real-websites/add`, …).

---

8. Security design points (good viva material)

Threat	Defence in this code
Rule bypass via odd URLs (`/%61dmin`, `//admin`, `/x/../admin`)	`normalize_path()`; proxy forwards the *normalised* path
HTTP request smuggling	`httpmsg` rejects both `Content-Length` + `Transfer-Encoding`, conflicting lengths, whitespace before `:`
Impersonating a user	Identity comes from the **TLS client certificate**, not a header the client supplies
Keyword evasion by splitting across packets	`StreamInspector` line re-assembly + hold-back of keyword prefixes
Compressed bodies hiding content	Proxy strips `Accept-Encoding` so bodies arrive uncompressed
Domain suffix trick (`notyoutube.com`)	match is `== domain` or `endswith("." + domain)`
Requester approving own request	`@role_required("admin")` on approve/reject/revoke
Cross-site request forgery	per-session CSRF token on every POST
The Browse page used to reach internal addresses (SSRF)	only `http`/`https`; the host must resolve to a **public** IP (`is_global`) so localhost, the admin console and the LAN are refused; every redirect hop is re-checked against the policy and the IP rule; login required
Scripts from a fetched page attacking the dashboard	the preview is a `sandbox` iframe with no `allow-scripts` and no `allow-same-origin`, so it has a throw-away origin (the only permission granted is `allow-top-navigation-by-user-activation`, i.e. a link the user clicks may navigate the page)
A forged certificate for the wrong site	the block page only issues certificates for valid host names (strict pattern, never a bare IP) and the CA is dedicated, locked down and removable
Resource exhaustion	`max_connections` semaphore, per-user caps, rate limit, header/body size limits, idle timeouts
Admin console abuse	listens on `127.0.0.1` only, token authentication, TLS
Bad policy edit taking the proxy down	edits are validated; invalid policy is **rejected** and the old version stays active

9. Honest limitations (examiners like candour)

1. **Hostname-only HTTPS filtering.** The proxy can't see URLs/paths/content inside HTTPS (it doesn't decrypt). Blocking `youtube.com` works; blocking a *page* on it doesn't. (The optional block page decrypts only **blocked** hosts, to show its message.)
2. **Bypassable.** Only traffic routed *through the proxy* is filtered. VPNs, DNS-over-HTTPS, or turning the proxy setting off bypass it. Real enforcement needs a gateway/firewall or a macOS Network Extension.
3. **Basic proxy auth is cleartext** and the demo passwords are public; Layer B also compares passwords with `==` (not constant-time). Fine on loopback, not for a shared network.

4. **File-based policy for Layer B** is re-read on every request: simple and live, but not scalable, and read-modify-write from multiple processes isn't locked (writes are atomic, but two simultaneous admin edits could race).
5. **Layer B revalidates open tunnels by polling** (once a second), whereas Layer A is pushed the change the moment a policy swaps. Fine for a demo, but a push model would scale better.
6. **Flask development server** — fine for a demo, not production.
7. `config/proxy.json` has `listen_host: 0.0.0.0` — Layer A is reachable from the LAN by default; restrict it for any shared setting. It also ships `require_client_cert: false` (clients without a certificate become `anonymous`) and a default admin token (`cn-proxy-admin`) — change both for shared use.

Possible improvements: TLS interception with a managed CA (to see paths), a database instead of JSON, hashed proxy credentials, push notifications for requests, SSO, per-request audit export.

---

10. Viva question bank (with model answers)

A. Basics

Q1. What does your project do? It filters network access through a proxy using ordered rules; a dashboard lets an admin manage rules and approve users' requests for temporary access, which expires automatically.

Q2. What is a proxy? Forward vs reverse? A middleman between client and server. A *forward* proxy acts for clients (the client is configured to use it) — mine. A *reverse* proxy sits in front of servers (load balancers, CDNs).

Q3. Why use a proxy for filtering rather than filtering in the browser? Central, enforced outside the user's control, works for any app that honours the proxy, and gives one place to log and audit.

Q4. What's the difference between your two proxies? `proxy.py` / `netproxy` is a TLS proxy for simulated lab services with identity, rule engine, content filters. `forward_proxy.py` is a standard HTTP/HTTPS proxy for real domains, hostname filtering only.

B. Networking / TLS

Q5. What is `CONNECT` and why is it used for HTTPS? It asks the proxy to open a raw TCP tunnel to `host:port`. The proxy can't parse HTTPS, so it just relays encrypted bytes after `200 Connection Established`.

Q6. Can the proxy see which page I visit on an HTTPS site? No — only the hostname (from `CONNECT`). The path and content are inside TLS.

Q7. What is mutual TLS and why use it? Both sides present certificates. The client's certificate Common Name becomes its identity, which can't be faked by editing a header. Without a certificate the user is `anonymous`.

Q8. What is a CA and what does `gen_certs.py` create? A Certificate Authority signs certificates. The script makes a CA, the proxy's server certificate and client certificates; clients verify the proxy (name `netproxy`) against the CA.

Q9. Why TCP_NODELAY? Disables Nagle's algorithm so small interactive messages (chat) aren't delayed.

Q10. What's a half-close and why do you handle it? A side finishes sending but can still receive. `shutdown_write()` passes this through so protocols like echo/upload complete correctly (`test_half_close_through_tunnel`).

Q11. Why does `Channel.try_recv` call `recv()` before `select()`? An `SSLSocket` may already hold decrypted data in its buffer; `select()` only sees the raw socket and would wait for data that has already arrived.

C. Policy engine

Q12. What does "first match wins" mean and why does order matter? Rules are tested top to bottom and the first whose conditions all match decides. So a temporary *allow* must sit above the *deny* to override it.

Q13. What is the default action and what's yours? What happens when no rule matches. Layer A: `deny` (whitelist). Layer B: `allow` (blacklist).

Q14. What can a rule match on? Users/groups (`@staff`, `*`, `anonymous`, with `except`), source network (CIDR), service, domain, path (glob), HTTP method, and a time schedule (days + hours, including windows crossing midnight).

Q15. Connect stage vs request stage? Connect: identity, IP, service, time known. Request: method and path also known. Path-scoped rules can only be finally decided per request; at connect time a deny is skipped and an allow admits provisionally.

Q16. Why is `Policy` immutable and swapped atomically? So a connection thread can never observe a half-updated rule list. Every change builds a new validated `Policy` (version+1) and swaps the reference under a lock; invalid changes are rejected.

Q17. How do rule changes apply without restarting? A watcher thread polls the file's mtime each second; admin commands use `store.mutate()`. After each swap `revalidate()` re-checks open connections and kills the ones now denied.

Q18. How does the schedule rule work? `Schedule.active(now)` compares minutes-since-midnight and weekday; if `start > end` the window crosses midnight and belongs to the day it starts on.

Q19. What is path normalisation and why is it needed? (see §4.6) Prevents `/%61dmin`, `//admin`, `/x/../admin` from evading `/admin/*`.

D. Content filtering

Q20. Difference between block, redact, log? Block refuses (403 or terminates the stream), redact replaces the matched text and continues, log only records. If a block matches it wins outright.

Q21. Why can't you just scan each `recv()` chunk for keywords? TCP has no message boundaries; a keyword can straddle two chunks. `StreamInspector` buffers into lines and holds back a tail that could begin a keyword.

Q22. What is the "fast path" in `_process`? One regex scan over the whole chunk; if nothing matches, forward it untouched. Only on a hit does it inspect line by line.

Q23. How do you stop executables being downloaded? A `mime` filter (F04) checks the response `Content-Type`; staff are exempt via `except_users`.

Q24. Why strip `Accept-Encoding`? A gzip-compressed body can't be scanned, so the proxy asks the server for identity encoding.

E. Concurrency & robustness

Q25. How does the server handle many clients? Thread per connection, capped by a `BoundedSemaphore`; non-blocking sockets with `select`.

Q26. How is a live session terminated? `kill()` sets a `threading.Event`; the relay loop checks it every ≤0.1 s and raises `Terminated`.

Q27. What are idle and handshake timeouts for? Stop slow/stuck clients (slowloris) from holding resources.

Q28. How are rate limits implemented? Per identity: current-connection counter plus a `deque` of recent connection times (sliding window); exceeding either returns `429`.

F. Dashboard & web security

Q29. How is the requester prevented from approving their own request? Server-side `@role_required("admin")` on the approve/reject/revoke routes (returns 403) — hiding the button isn't security.

Q30. What is CSRF and how do you prevent it? A malicious site making your browser submit a form to the dashboard. Each session has a random token that every POST must include; compared with `hmac.compare_digest`.

Q31. How are passwords stored? Salted PBKDF2-SHA256 hashes via Werkzeug, never plaintext; verified with `check_password_hash`.

Q32. Why did the app crash with `hashlib has no attribute 'scrypt'` and how was it fixed? Werkzeug's default hash is scrypt, which this Python build (linked to LibreSSL) lacks. I set `method="pbkdf2:sha256"`.

Q33. Why bind to 127.0.0.1? The dashboard is an admin interface using demo credentials; keeping it on loopback avoids exposing it to the network.

Q34. Why write JSON via temp-file + `replace()`? Atomic — the proxy reading the file at the same moment never sees a half-written document.

G. The temporary-unblock feature

Q35. Walk me through requesting a site. (Use §6 diagram.) Request → admin approves → allow rule with `expires_at` inserted at top → proxy matches it → expiry makes it inert.

Q36. How does a site "re-block" itself after expiry? `match_rule` skips rules where `rule_expired()`; the older deny underneath matches again. Enforced in the data path, no job needed.

Q37. What happens to a tunnel that is open when the time runs out — or when access is revoked? `tunnel()` wakes every second, stops at the grant's `deadline`, and re-runs `match_rule` for the tunnel's host and user, so a revoke or a new deny closes it within about a second.

Q38. Why insert at index 0 instead of appending? First-match-wins: appended, the older deny would match first and the grant would never apply.

Q39. How do you stop a user requesting a site that isn't blocked? `blocked_sites_for(user)` uses the proxy's own `match_rule`; only domains it currently denies for that user are accepted.

Q40. Does approval affect other users? No — the rule's `users` is `["bob"]`; Carol is still blocked (tested).

H. Testing

Q41. How did you test it? Unit tests for the rule engine, filters, and domain matching; integration tests that start the whole stack over real sockets (TLS identity, denials, path rules, hot reload, admin kill, rate limits); role tests; and tests for roles (self-approval refused), the block-page request link, and the Browse page's SSRF guards. In total 86 tests; run with `venv/bin/python -m unittest discover -s tests`.

Q42. One known failing test? `test_tls_and_identity` expects TLS 1.3; on this Mac the system Python negotiates TLS 1.2. It's an environment difference, not a logic bug. It is the only failure in the suite (85 of 86 pass).

I. "What if…" / critical thinking

Q43. What if the user turns off the proxy setting? Filtering is bypassed — a proxy only filters traffic sent to it. Real enforcement needs network-level controls.

Q44. Can the proxy block a single YouTube video? No for HTTPS — it only sees `youtube.com`. It would need TLS interception.

Q45. What if two admins edit at once? Writes are atomic but read-modify-write isn't cross-process locked; a database or file lock would fix it.

Q46. What if the policy file is invalid? Rejected with an error; the previous version keeps running.

Q47. How would you scale this? Replace JSON with a database/cache, use async I/O or a process pool, central logging, and hashed credentials/SSO.

Q48. Why simulated services? A safe, self-contained lab: reproducible demos and tests without touching real sites.

J. Block page, Browse and YouTube

Q49. Why does a blocked HTTPS site show "site can't be reached" instead of a message? The browser asks for a tunnel (`CONNECT`) before it starts TLS. When the proxy refuses, browsers discard the refusal's body and show their own error page.

Q50. How does your block page work for HTTPS? With `--block-page` the proxy answers the `CONNECT` with `200` and completes TLS itself, using a certificate for that host signed by a dedicated local CA (made with `openssl`). If the browser trusts the CA, it shows our page — the reason, who restricted it, and a request button. Only blocked hosts are intercepted.

Q51. Isn't that dangerous? What are the risks and how do you limit them? While trusted, the CA key could sign certificates for any site on that machine. Mitigations: a dedicated CA, key `chmod 600` and git-ignored, short-lived (30-day) leaf certificates, strict host validation (no IPs), only blocked hosts intercepted, and the trust is removed after the demo.

Q52. Why did YouTube show "Connect to the Internet" with a Downloads page when blocked? YouTube's **service worker** serves a cached offline page when the network fails. The block page unregisters that worker and clears the origin's caches and storage (`Clear-Site-Data` plus a script), so the offline page and its downloads can't open. Cookies are kept.

Q53. Why did YouTube keep playing after you revoked access? Two reasons, both fixed. Video streams from `googlevideo.com`, which wasn't in the rule — now `DOMAIN_FAMILIES` makes `youtube.com` cover its related domains. And revoking didn't close tunnels that were already open — now each tunnel re-checks its rule every second and closes within about a second of a revoke.

Q54. The Browse page fetches URLs for the user — how do you stop it being abused to reach internal addresses? Only `http` / `https`; the host must resolve to a public IP (so localhost, the admin console and the LAN are refused); redirects are followed manually and every hop is re-checked against the policy and the IP rule; the preview runs in a sandboxed, scripts-off frame; login is required.

---

11. Demo script (3 minutes)

1. Start: `python3 services.py` · `python3 proxy.py` · `python3 forward_proxy.py --block-page` · `python3 app.py` (each in its own terminal, venv active). Trust the block-page CA once (command printed on first start) and set the macOS web proxies to `127.0.0.1:8080` — see `README_ACCESSWINDOW.md`.
2. Admin (`admin` / `admin-demo`) → **Website access** → add `youtube.com`, **Block**, all users.
3. In the browser open `youtube.com` (proxy login `bob` / `bob-demo`) → the **block page**: why, who, when. (Or `curl -x http://127.0.0.1:8080 --proxy-user bob:bob-demo https://youtube.com/ -I` → 403.)
4. Click **Request temporary access** on the block page → sign in as `bob` → the request form opens with youtube.com selected → 5 minutes, reason.
5. Admin → **Access requests** → **Approve**. Reload YouTube → it opens. Show **Temporary grants** counting down. Also show **Browse** → `youtube.com` (blocked card) and `example.com` (allowed preview).
6. Wait / **Revoke** → the open YouTube connection is cut within a second and the block page returns. Show the **Activity log** and the blocked-site alerts.
7. Open `packet_flow.html` and narrate: *packet → auth → target → first-match rule → verdict → tunnel or 403.*

12. Cheat sheet — numbers & names

Thing	Value
Dashboard	`127.0.0.1:5000`
Forward proxy (real domains)	`127.0.0.1:8080`
TLS proxy / admin console	`:8443` / `127.0.0.1:9443`
Simulated services	web 9001 · chat 9002 · echo 9003 · files 9004 · bulk 9005
Demo users	`admin`, `bob`, `carol` (`<name>-demo`)
Policy files	`config/policy.json` (Layer A) · `config/real_website_policy.json` (Layer B)
Logs	`logs/access.log` (A) · `logs/real_proxy_access.jsonl` (B)
Durations offered	5 / 15 / 30 / 60 minutes
Block page	`python3 forward_proxy.py --block-page` · CA at `certs/blockpage/ca.crt` · remove trust: `security delete-certificate -c "AccessWindow Block Page CA" …`
Related domains	`youtube.com` ⇒ `googlevideo.com`, `ytimg.com`, `youtu.be`, `youtube-nocookie.com`
Tunnel re-check	every 1 s (revoke / new deny closes open tunnels)
Sample Layer A rules	R01 staff allow · R02 quarantined net deny · R03 anonymous can't use chat/files · R04 `/admin` deny · R05 `/social` deny Mon–Fri 09–17 · R06 students allow · R07 guests web+echo
Sample filters	F01 gambling block · F02 card numbers redact · F03 "confidential" block (non-staff) · F04 executables block · F05 profanity redact · F06 "password" log